

Kanbien Modular Monorepo Entity Builder

Strict layer ownership structure

A modular monorepo with strict layer ownership.

Table of Contents

1. Repo root
2. packages/design-system
3. packages/frontend-kit
4. platform/
5. packages/core
6. apps/
7. harness/
8. tools/build
9. Persistence
10. Infra
11. The core entity flow

1. Repo root

```
kanbien/  
  apps/  
  packages/  
  platform/  
  infra/  
  harness/  
  tools/  
  docs/
```

2. packages/design-system

Owns all visual primitives.

```
packages/design-system/  
  tokens/  
    foundations/  
    builders/  
    resolved/  
  primitives/  
  patterns/  
  components/  
  resolvers/
```

Rule: only the design-system harness layer can edit this. Apps consume it only.

Flow:

design label -> foundation tokens -> builder tokens -> primitives -> patterns -> app UI

3. packages/frontend-kit

Portable React feature shell.

```
packages/frontend-kit/  
  entity-pages/  
  crud-layouts/  
  forms/  
  tables/  
  navigation/
```

This knows how to render entities, but not what a User or Invoice is.

4. platform/

Thin runtime layer.

```
platform/  
  server/  
    http.ts  
    cors.ts  
    rate-limit.ts  
    auth.ts  
    routes.ts  
  mount.ts
```

Features/apps register through:

```
mount(app)
```

Keep this boring.

5. packages/core

Reusable backend capabilities.

```
packages/core/  
  jobs/  
  files/  
  auth/  
  audit/  
  notifications/  
  validation/  
  persistence/
```

No business features here.

6. apps/

Actual product code.

```
apps/crm/  
  entities/  
    user/  
      entity.schema.ts  
      rules.ts  
      permissions.ts  
      ui.ts  
      api.ts  
      tests/  
  features/  
  migrations/  
  app.mount.ts
```

Each app can run alone, but can also mount into the platform.

7. harness/

The entity builder brain.

```
harness/  
  entity-definition/  
  prompts/  
  validators/  
  generators/  
  contracts/  
  layer-rules/
```

This captures:

entity -> fields -> rules -> permissions -> workflows -> persistence -> UI config

Then emits deterministic code.

8. tools/build

Code generation only.

```
tools/build/  
  generate-entity.ts  
  generate-crud.ts  
  generate-migration.ts  
  generate-tests.ts
```

Important: generation should be idempotent. Running it twice should not bloat the repo.

9. Persistence

Use central migration ownership per app:

`apps/crm/migrations/`

Entity folders can define desired persistence shape, but migrations should be generated into one ordered app migration stream.

Use Prisma, Drizzle, or Kysely. My bias: Drizzle + SQL migrations.

10. Infra

```
infra/  
  terraform/  
    aws/  
      network/  
      app/  
      database/  
      secrets/  
      ci-cd/
```

Deployment should consume app manifests:

```
apps/crm/app.manifest.ts
```

11. The core entity flow

chat

- > harness captures entity definition
- > validates contract
- > generates backend schema/api/tests/migration
- > generates frontend config
- > mounts feature into app
- > platform mounts app
- > CI runs checks
- > Terraform deploys staging
- > promote to prod

The biggest correction: don't let features own everything. Let entities define intent, generators produce implementation, apps own business composition, and platform own runtime concerns.